

AI 技术应用场景匹配报告

一、引言

随着生成式人工智能（Generative AI）、大语言模型（Large Language Model, LLM）、智能体（Agent）等技术的爆发式发展，软件工程正从传统的流程驱动模式，向人机协同的智能化范式加速转型。AI 技术凭借其在内容生成、模式识别、知识推理与自动化执行等方面的能力，正逐步渗透到软件开发生命周期的各个阶段，为软件过程优化提供了新的可能。

本报告基于前期完成的《网页购物系统软件过程定义文档》（以下简称“原过程文档”）。该文档参考 ISO/IEC 12207、ISO/IEC 15504 等标准，结合四人本科生小组、8-12 周短周期项目的实际情况，制定了简化的瀑布+迭代混合模型，包含以下五个主要过程：

过程编号	过程名称	持续时间	能力等级目标
REQ-01	需求获取与分析	第 1 周	Level 2
DES-01	系统设计	第 2 周	Level 2
IMP-01	软件实现	第 3-8 周（约 6 周）	Level 2
TST-01	软件测试	第 9-10 周（约 1.5-2 周）	Level 2
DPL-01	部署与维护	第 11-12 周（约 1-2 周）	Level 1

本报告从上述五个过程中筛选出 3-5 个 AI 技术可显著提升效率的关键环节，进行后续软件过程改进方案设计。

二、关键环节筛选与匹配分析

综合考虑技术成熟度、实施成本、预期收益以及四人小组、短周期项目的特点，从五个过程中筛选出以下 4 个**关键环节**作为 AI 技术重点改进对象：

优先级	原过程编号	关键环节	筛选理由
★★★	TST-01	测试验证	时间紧凑（1.5-2 周），AI 可快速产出测试资产

优先级	原过程编号	关键环节	筛选理由
★★★	IMP-01	编码实现	占比最大（6周），AI代码生成工具成熟
★★☆	REQ-01	需求分析	时间紧张（1周），AI可提速文档与建模
★★☆	DES-01	系统设计	设计质量影响全局，AI可辅助数据库与接口设计

以下对这四个关键环节进行详细的 AI 应用场景匹配分析。

（一）关键环节一：测试验证（TST-01）

1. 原过程痛点分析

原过程文档中，测试阶段分配 1.5-2 周，主要活动包括：

- 设计测试用例（重点覆盖核心功能）
- 执行功能测试、集成测试、基本兼容性测试
- 使用 GitHub Issues 记录缺陷
- 缺陷修复与回归测试

存在的主要问题：

- 手工设计测试用例耗时，且容易遗漏边界条件和异常场景
- 回归测试手工执行效率低，重复劳动多
- 测试报告编写占用宝贵时间
- 四人小组中测试负责人往往由开发人员兼任，测试经验有限

2. AI 应用场景

场景	具体描述	推荐工具
测试用例自动生成	根据需求文档和接口设计，AI 自动生成覆盖功能、边界、异常场景的测试用例表	ChatGPT、通义千问
测试脚本自动生成	AI 根据测试用例生成自动化测试脚本（如 Postman 集合、Jest 单元测试）	Postman AI、Copilot
缺陷智能分析	AI 分析测试日志，自动归类失败用例并推测可能原因	ChatGPT、Claude
回归测试优化	AI 根据代码变更范围推荐需要执行的回归测试子集	智能测试平台

3. 预期收益

指标	传统方式	AI 辅助方式	提升效果
测试用例编写时间	3-4 天	1-1.5 天	减少 60%以上
回归测试执行时间	2-3 天	0.5-1 天	减少 60%以上
缺陷发现率	基准	+30%	显著提升
测试报告编写	1 天	0.5 天	减少 50%

（二）关键环节二：编码实现（IMP-01）

1. 原过程痛点分析

原过程文档中，实现阶段占据 6 周时间，是项目的核心与主体，主要活动包括：

- 前端页面开发（Vue）
- 后端接口开发与数据库实现
- 前后端接口联调
- 编写简单单元测试代码
- 代码互审（Pull Request）

存在的主要问题：

- 大量重复性 CRUD 模板代码编写耗时
- Bug 产生率较高，后期修复成本大
- 代码互审依赖人工，效率有限且可能遗漏问题
- 单元测试编写容易被忽视或质量不高

2. AI 应用场景

场景	具体描述	推荐工具
代码自动生成	根据注释或函数签名自动生成 CRUD 代码、前端组件、数据库访问代码	GitHub Copilot、Claude code
智能代码补全	开发过程中实时提供代码建议，加快编码速度	GitHub Copilot
AI 代码审查	PR 时自动识别代码异味、潜在 Bug、性能问题，给出优化建议	Claude、Copilot
单元测试生成	根据业务代码自动生成单元测试用例，提高测试覆盖率	Copilot

3. 预期收益

指标	传统方式	AI 辅助方式	提升效果
编码效率	基准	+40%-60%	显著提升
CRUD 代码编写时间	2-3 周	0.5-1 周	减少 70%以上
单元测试覆盖率	约 75%	85%以上	提升 10%以上
代码审查效率	基准	+50%	显著提升

(三) 关键环节三：需求分析（REQ-01）

1. 原过程痛点分析

原过程文档中，需求阶段仅分配 1 周时间，主要活动包括：

- 小组讨论确定项目范围和功能列表
- 简单竞品分析（参考淘宝、京东）
- 识别功能需求和基本非功能需求
- 绘制简单用例图 and 业务流程图
- 编写简易需求规格说明（SRS）

存在的主要问题：

- 四人小组讨论易遗漏边缘需求（如支付异常、库存扣减并发等）
- 需求文档编写耗时，挤占后续设计时间
- 手动绘制用例图 and 业务流程图效率较低
- 非功能需求（性能、安全）容易考虑不周

2. AI 应用场景

场景	具体描述	推荐工具
竞品分析辅助	AI 自动搜索总结淘宝、京东等电商平台的核心功能，生成功能对比清单	ChatGPT、Perplexity
需求自动整理	AI 分析小组讨论记录，自动提取功能需求，基于电商领域知识补全常见遗漏	Deepseek
用户故事生成	AI 根据功能清单自动生成符合 INVEST 原则的用户故事	ChatGPT
用例图自动生成	AI 根据需求文档自动生成 Mermaid 格式的用例图代码	ChatGPT

3. 预期收益

指标	传统方式	AI 辅助方式	提升效果
需求整理效率	基准	+50%以上	显著提升
SRS 编写时间	3-4 天	1.5-2 天	减少 50%

指标	传统方式	AI 辅助方式	提升效果
需求遗漏率	基准	-30%	明显改善
用例图绘制时间	0.5-1 天	0.5-1 小时	减少 80%以上

（四）关键环节四：系统设计（DES-01）

1. 原过程痛点分析

原过程文档中，设计阶段仅分配 1 周时间，主要活动包括：

- 技术选型（前后端分离架构）
- 数据库设计（ER 图、表结构）
- 接口设计
- 模块划分与简单架构设计

存在的主要问题：

- 四人小组技术经验有限，技术选型可能存在不合理之处
- 数据库表结构设计易出现冗余或遗漏字段（如订单表缺少状态字段）
- API 接口设计文档编写耗时
- ER 图和架构图手动绘制效率低

2. AI 应用场景

场景	具体描述	推荐工具
数据库设计辅助	输入功能需求，AI 自动生成符合第三范式的 ER 图及建表 SQL 脚本	ChatGPT、DeepSeek
API 接口设计	AI 根据功能需求自动生成 RESTful API 接口清单，含请求/响应格式	ChatGPT
技术选型建议	AI 根据项目约束推荐合适技术栈并对比优缺点	Claude、GPT-4

场景	具体描述	推荐工具
架构图生成	AI 根据技术选型和模块划分生成系统架构图代码	ChatGPT (Mermaid)

3. 预期收益

指标	传统方式	AI 辅助方式	提升效果
数据库设计效率	基准	+60%	显著提升
API 设计时间	2-3 天	1 天	减少 60%
设计文档编写	基准	-50%	显著减少
设计缺陷率	基准	-25%	明显改善

三、筛选结果总结

综合以上分析，从原软件过程的五个阶段中筛选出 4 个 AI 技术可显著提升效率的关键环节，汇总如下：

排名	关键环节	原过程编号	预期效率提升
1	测试验证	TST-01	测试用例编写时间减少 60%以上
2	编码实现	IMP-01	编码效率提升 40%-60%
3	需求分析	REQ-01	SRS 编写时间减少 50%
4	系统设计	DES-01	数据库设计效率提升 60%

说明：

- 测试验证与编码实现被列为最高优先级，原因如下：
 - 编码阶段占据 6 周，是项目时间占比最大的环节，AI 代码生成工具已较为成熟
 - 测试阶段时间紧凑（仅 1.5-2 周），AI 可快速生成测试用例与脚本，缓解人力压力

- **需求分析与系统设计**虽时间较短，但作为上游环节，其质量直接影响后续开发效率，同样具有较高改进价值

四、总结

上述四个环节涵盖了软件开发生命周期的核心阶段，AI 技术的引入有望在有限的人力资源下，显著提升整体开发效率与产出质量，特别适合四人小组、8-12 周短周期项目的实际场景。

后续工作将基于本报告的筛选结果，重点针对**测试验证**与**编码实现**两个最高优先级环节，进一步设计 AI 与传统软件过程的融合方案，并选择其中一个核心场景开展原型验证，使用真实 AI 工具进行实际测试，对比传统方式与 AI 辅助方式的效率与质量差异。